

Konkani-Marathi Speech Pipeline & Dialect Normalisation

Architectural Review, Technical Blueprint & Implementation Plan

Aditya Kinjawadekar

July 25, 2026

Document Crux: Diarization → Dialect Normalization → ASR

This document serves as the technical blueprint for the Konkani-Marathi speech recognition pipeline. It begins with an architectural review evaluating audio-level dialect normalization prior to speech recognition versus text-level cleanup, identifying key technical challenges, intermediate acoustic feature adaptation, and direct model fine-tuning paths. The subsequent sections provide the complete data strategy, model architecture (TokAN adaptation), synthetic parallel data pipeline, and execution roadmap.

1. Thesis Defense: Case FOR Standalone Normalisation vs. Case AGAINST Alternatives

When proposing a standalone dialect normalisation component (whether a full audio-to-audio converter or a lightweight acoustic feature adapter), a thesis defense panel will scrutinize why a standalone module is necessary rather than fine-tuning the downstream ASR model (e.g., using LoRA adapters) or cleaning up text after transcription. This section establishes the rigorous academic, architectural, and information-theoretic case FOR a standalone normaliser and AGAINST all alternatives.

1.1. The Thesis Core Defense Statement

Thesis Core Defense:

We propose building a **standalone, decoupled dialect normalisation component** that transforms regional non-standard speech (or its latent acoustic features) prior to downstream recognition. We explicitly defend this against fine-tuning downstream ASR models via LoRA adapters or patching transcripts post-ASR. Standalone normalisation guarantees **universal model agnosticism**, enables **black-box API deployment**, prevents **catastrophic forgetting**, preserves **audio-level intelligibility for multi-task applications**, and disentangles **phonological shift from language modeling**.

1.2. The Case FOR a Standalone Normalisation Component

1. **Decoupled Modularity & Universal Model Agnosticism (Plug-and-Play Architecture):** A standalone normaliser operates directly on speech waveforms or model-agnostic acoustic representations ($X_{\text{Konkani}} \rightarrow \hat{X}_{\text{Marathi}}$). It functions as a universal upstream adapter. Once trained, it can precede **ANY downstream ASR engine** (IndicConformer, Whisper-v3, Meta MMS, Google STT), human listeners, live translation systems, or biometric speaker verification pipelines without modifying downstream code or weights.

2. **Real-World Black-Box API Compatibility:** In commercial and enterprise deployments (healthcare, public infrastructure, customer support), ASR systems are black-box commercial APIs (e.g., Google Cloud Speech-to-Text, Azure Speech) accessed strictly over network endpoints. Internal model weight fine-tuning (LoRA) is **impossible**. A standalone front-end normaliser is the **only viable mechanism** to enable black-box APIs to transcribe unsupported regional dialects.
3. **100% Preservation of Pre-Trained ASR Knowledge (Zero Catastrophic Forgetting):** Fine-tuning a 600M+ parameter ASR model on a small, low-resource regional dataset risks corrupting its pre-trained linguistic, semantic, and language modeling representations built on tens of thousands of hours of data. A standalone normaliser leaves the downstream ASR model 100% frozen, preserving its pre-trained robustness.
4. **Multi-Task Usability & Audio Output Retention:** Direct ASR fine-tuning destroys the audio signal, producing only text strings. A standalone audio/feature normaliser preserves or reconstructs the speech signal, enabling human cross-dialect listening (standard Marathi speakers understanding regional Konkani speech in real time), voice dubbing, speech-to-speech translation, and emotion preservation.
5. **Disentanglement of Phonology vs. Language Modeling:** Dialect variation is fundamentally an **acoustic-phonological transformation**, not a structural semantic change. Conflating acoustic adaptation with sequence-to-sequence language modeling inside an ASR model forces the decoder to solve two distinct problems simultaneously. Disentangling normalisation into a specialized front-end isolates the phonological shift problem and allows explicit optimization for speaker identity preservation, acoustic naturalness (UTMOS), and phone contrast enhancement.

1.3. The Case AGAINST Direct LoRA Fine-Tuning of Downstream ASR

1. **Tightly Coupled Model Dependency:** LoRA adapters ($W'_{ASR} = W_{ASR} + BA$) bind the adaptation strictly to a single specific model architecture and checkpoint. Upgrading the underlying ASR model renders the LoRA weights useless, requiring costly re-training and re-collection of dialect data.
2. **Inapplicability to Black-Box Endpoints:** LoRA requires open access to internal weight matrices, rendering it completely un-deployable over third-party APIs or proprietary hardware accelerators.
3. **Destruction of Audio Intelligibility:** LoRA fine-tuning provides zero benefit to non-ASR tasks. It cannot produce audible speech for human cross-dialect understanding or live speech translation.
4. **Combinatorial Explosion across Dialects:** Fine-tuning via LoRA requires maintaining separate adapter modules for every sub-dialect (Goan Konkani, Mangalorean Konkani, Malvani, Chitpavani). In contrast, a standalone normaliser uses a shared discrete acoustic codebook (e.g., TokAN VQ-VAE) to map arbitrary dialect tokens onto standard acoustic primitives.

1.4. The Case AGAINST Post-ASR Text Normalisation (NMT / LLM Rewriting)

1. **Fatal Information Loss Under CTC / Argmax Tokenization:** When non-standard Konkani speech is processed by a Marathi ASR, acoustic frames mismatching Marathi distributions get misclassified into incorrect Devanagari subwords during CTC/beam-search decoding:

$$Y^* = \arg \max_Y P(Y \mid X_{\text{audio}})$$

The moment speech is tokenized into text Y^* , **acoustic phoneme information is irreversibly destroyed**. No downstream text model (NMT or LLM) can reliably guess whether a transcribed character was what the speaker said or a phonetically garbled mishearing.

2. **High-Confidence Hallucination Propagation:** ASR decoders output misheard tokens with high softmax confidence. Text rewriting models process these corrupted tokens as ground truth, leading to catastrophic semantic hallucinations.

1.5. Defense Summary Matrix: Case-FOR vs. Case-AGAINST

Architecture	Core Mechanism	Case FOR (Strengths)	Case AGAINST (Fatal Flaws)
Standalone Audio/Feature Normaliser (Proposed Thesis)	Pre-ASR acoustic feature or waveform transformation ($X_{L2} \rightarrow \hat{X}_{L1}$)	Universal model agnosticism; works with black-box APIs; zero catastrophic forgetting; preserves audio for human listening	Requires acoustic pair generation or discrete token alignment
Direct LoRA ASR Fine-Tuning	Parameter-efficient fine-tuning of ASR weights ($W_{ASR} + \Delta W$)	Parameter efficient (<1% weights fine-tuned); high performance on target model	Tightly coupled to 1 model checkpoint; cannot work on black-box APIs; destroys audio signal
Post-ASR Text Normalisation	Post-processing garbled text via NMT/LLM rewriting	Easy text-only pair collection	Fatal flaw: Cannot recover acoustic phonemes lost during ASR argmax tokenization

2. Review of the Planned Konkani Speech Pipeline

2.1. What This Pipeline Is Proposing

The core architecture proposes a modular, three-stage cascade for transcribing regional Konkani speech using an existing high-resource Marathi speech-recognition (ASR) system:

1. **Stage 1 — Diarization (Speaker Segmentation & Clustering):** Processes raw, multi-speaker continuous audio streams to detect active speech boundaries and assign speaker labels, partitioning the input waveform into homogeneous single-speaker audio segments $A_1, A_2, \dots, A_K$.
2. **Stage 2 — Audio Dialect Normalisation / Acoustic Adaptation:** Transforms the Konkani acoustic signal A_i directly in the signal or feature space into an adapted representation $\hat{A}_i$ calibrated to match the phonetic and acoustic expectations of the target Marathi ASR model.
3. **Stage 3 — ASR Transcription (Marathi Acoustic Model):** Feeds the adapted acoustic representation $\hat{A}_i$ into a pre-trained Marathi ASR backbone (e.g., IndicConformer-600M or IndicWhisper) to emit final Devanagari text transcripts Y_i .

The Fundamental Architectural Shift: The central design innovation here, compared to conventional post-ASR text cleanup (NMT/LLM rewriting), is that normalisation is applied **pre-transcription directly in the acoustic domain**, rather than post-transcription on garbled text strings.

2.2. Diarization: Technical Execution & Language Isolation

Diarization operates strictly on acoustic signal characteristics (pitch, formant structures, spectral envelope, speaker embeddings) and VAD (Voice Activity Detection) timestamps, independent of linguistic semantics or underlying language phonology.

- **Execution Workflow:** Raw audio passes through a neural VAD (e.g., Pyannote-Audio 3.1) to trim silence. Active speech frames are projected into a metric space using speaker encoders (e.g., ECAPA-TDNN or x-vectors). Agglomerative Hierarchical Clustering (AHC) or Variational Bayesian HMM (VBx) groups vectors into distinct speaker IDs.
- **Isolation Guarantee:** Because diarization functions purely at the acoustic identity layer, it does not interact with the dialect discrepancy or language boundary problem. Placing it first is uncontroversial, correct, and isolates downstream processing to clean single-speaker segments.

2.3. Dialect Normalisation on Audio: Acoustic Root Cause Analysis

This is the most critical conceptual upgrade compared to a text-based cleanup approach. Fixing speech at the audio/feature stage targets the actual root cause of recognition failure rather than attempting to patch downstream symptoms.

Information Loss in ASR Tokenization: When an ASR model processes acoustic frames, it performs non-linear projection into subword/character token probabilities via CTC beam search or autoregressive decoding:

$$Y^* = \arg \max_Y P(Y \mid X_{\text{audio}})$$

If Konkani phoneme sequences differ from the Marathi acoustic model’s training distribution, the acoustic model misclassifies the phoneme frames into incorrect Marathi character tokens.

The moment the audio is projected into text tokens Y_{raw} , **the underlying acoustic information is irreversibly destroyed** (entropy loss under argmax projection). No downstream text-cleanup model (NMT or LLM) can reliably distinguish whether a transcribed token was what the speaker actually said or a phonetically garbled mishearing. By normalising at the audio/feature stage ($X_{\text{Konkani}} \rightarrow \hat{X}_{\text{Marathi}}$), the pipeline preserves phonemic contrast before irreversible tokenization occurs.

2.4. Where the Difficulty Actually Lies: Technical Options

While pre-ASR normalisation is theoretically superior, deploying it introduces technical choices evaluated below:

1. **Acoustic Conversion Complexity:** Disentangling speaker identity S , phonetic content C , and dialect accent A ($X_{\text{audio}} = g(S, C, A)$).
2. **Target Acoustic Ambiguity:** Full waveform voice conversion vs latent feature space mapping.
3. **Matched Training Data Bottleneck:** Sourcing parallel audio pairs via voice-cloning TTS.
4. **Acoustic Feature Adaptation (Light Front-End):** Operating directly on intermediate Log-Mel filterbanks or self-supervised HuBERT embeddings: $\hat{F}_{\text{Marathi}} = f_{\theta}(F_{\text{Konkani}})$.

2.5. Overall Verdict & Implementation Path

Strategic Verdict & Technical Path:

1. **Immediate Target (Phase 1):** Start with a lightweight **acoustic feature adaptation front-end** or ASR encoder adapter rather than a full audio-to-audio voice conversion system. This avoids the ambiguity of defining a full ‘normalized speech’ waveform target and executes within modest data and GPU budgets.
2. **Durable Target (Phase 2):** Treat all Konkani audio data collected during Phase 1 as the seed dataset for **direct fine-tuning of the Marathi ASR acoustic model** via LoRA adapters. Direct fine-tuning is the long-term, high-ceiling solution.

In short: Keep the three-stage order (**Diarization** → **Dialect Normalisation** → **ASR**), treat normalisation as a lightweight feature adapter, and plan from day one to transition to direct acoustic model fine-tuning.

3. Technical Blueprint: The Big Picture & TokAN Architecture

No one has built dialect normalisation for any Indian language yet. That’s what we’re doing.

The idea: take speech with a regional dialect (L2), and convert it to sound like the standard prestige variety (L1) — while keeping the original speaker’s voice. Same task as English dialect normalisation, but for Indic languages where regional dialects are spoken by native speakers, not L2 learners.

TokAN [2] is a recent state-of-the-art accent/dialect normalisation system for English. We’ll use it as our starting point. (*Verify its exact reported WER numbers and venue against the primary source before citing.*)

Target Venue and Contributions

Target Venue

Primary target: **IEEE/ACM Transactions on Audio, Speech and Language Processing (TASLP)** or **Computer Speech and Language (CSL)**. Conference precursor: submit a system-description version to **INTER-SPEECH 2027** or **ICASSP 2027** first, use reviewer feedback to strengthen the journal version.

Explicit Contributions

1. **First dialect normalisation system for any Indic language.** No prior work has built an end-to-end DN pipeline for Marathi, Hindi, or any Indian language. This is a first-mover contribution.
2. **Synthetic parallel data generation pipeline.** A reproducible, automated method using a voice-cloning TTS (real L1 audio + cloned-voice L2 synthesis) and RESPIN/OpenSLR 103 transcripts to create (real-L1, synthetic-L2) parallel speech pairs. No parallel human recording needed. Applicable to any language with a voice-cloning TTS and a dialect label source.
3. **Evaluation benchmark for Marathi dialect normalisation.** A publicly released evaluation protocol using RESPIN Marathi / OpenSLR 103: WER-before/after methodology, dialect classifier setup, and evaluation scripts. The benchmark itself is a contribution, citable by future work regardless of model choice.

4. **Multi-dialect, multi-language results.** Results across Marathi dialect pairs (Varhadi $\rightarrow$ Standard Pune, South Konkan $\rightarrow$ Standard Pune, North Konkan $\rightarrow$ Standard Pune) and Hindi (Awadhi $\rightarrow$ Khariboli). The comparison across dialects tells a story about which phonological contrasts are harder to normalise and why.
5. **Ablation study validating each architectural component.** VQ tokenizer vs direct spectrogram, with/without GRPO RL, synthetic-only vs synthetic+real training data, different codebook sizes. Every design choice justified individually.

Reference: TokAN Architecture

How It Works

1. **VQ tokenizer** trained jointly on L1 and L2 speech with shared codebooks
2. **Autoregressive encoder-decoder** does token-to-token conversion (L2 $\rightarrow$ L1)
3. **GRPO reinforcement learning** fine-tunes with WER + dialect classifier rewards
4. **Flow-matching synthesiser** conditioned on source speaker embedding (ECAPA-TDNN)

Results (7 English Accents)

WER dropped from 12.40% $\rightarrow$ 9.89% (SFT) $\rightarrow$ 9.23% (+RL).

What We Keep vs What Changes

What we keep from TokAN:

- Source-speaker conditioning via ECAPA-TDNN
- GRPO RL post-training
- Flow-matching duration predictor (optional)

What's different: *will decide later*

Which Language to Start With?

We go where the data is. Specifically, where the **L2 data** (dialectal speech with labels) is available.

L2 Data: What Exists (Freely Available, Dialect-Annotated)

Lang.	Dataset	Hours	Dialect Labels	Access
Marathi	RESPIN-S1.0 [12]	$\sim$ 1,200	4 dialects (Konkan South, Konkan North, Standard Pune, Varhadi)	GitHub

Lang.	Dataset	Hours	Dialect Labels	Access
	OpenSLR 103 [9]	99	3 sociolect groups	Direct
	LDC-IL Marathi	89	4 regions (Marathwada, Puneri, Vidarbha, Goa)	Paid
Hindi	RESPIN-S1.0 [12]	~2,500	5 dialects (Khariboli, Awadhi/Bundeli, Braj/Kanauji, Marwari, Garhwali)	GitHub
	LAHAJA [19]	12.5	83 districts	HF (test only)
	IndicVoices [5]	23.7K total	District/state metadata	HF
	VAANI [20]	31K+	165 districts	HF
	Nirantar	3,240	208 districts	Form-based

Why RESPIN-S1.0 is the primary L2 source. It is the only large-scale dataset with explicit dialect labels for both Marathi (4 dialects) and Hindi (5 dialects). Total 10K+ hours across 9 languages and 38 dialects. Dataset and code at <https://spiredatasets.ee.iisc.ac.in/respincorpus> and https://github.com/labspire/respin_baselines.git. NeurIPS 2025.

Why Marathi first. RESPIN provides ~1,200h of dialect-labelled Marathi speech across 4 distinct dialect groups (Konkan South, Konkan North, Standard Pune, Varhadi), supplemented by OpenSLR 103 (99h) and LDC-IL Marathi (89h). Marathi is our primary target due to its structural clarity and direct relevance to Konkani speech adaptation. Hindi (~2,500h across 5 dialects) serves as our secondary evaluation target. Our primary Marathi target is Varhadi (D4) / South Konkan (D1) → Standard Pune (D3 / IndicVoices-R Marathi).

This doc uses Marathi as the primary running example. Hindi is the second target. The pipeline is language-agnostic.

Marathi Dialect Landscape

Marathi is an Indo-Aryan language with strong regional dialectal variation across Maharashtra and Goa. Standard Marathi (Puneri / Pune variety) contrasts with several major regional varieties:

Dialect	Region	Key Differences from Standard
Standard (Pune)	Pune / Urban MH	Standard prestige variety — benchmark target
Varhadi	Vidarbha (E. MH)	Retroflex lateral [ɭ] → [j], <i>-le</i> case marker (vs <i>-lā</i>), ergative alignment
South Konkan	Sindhudurg, Ratnagiri	Coastal phonology, direct affinity with Konkani, vowel shifts, unique affixes
North Konkan	Palghar, Thane	Distinct palatal affricates, nasalisation loss, vocabulary overlap
Marathwada	Central MH	Verbal inflections, medial aspiration deletion

L2 Dialect Pairs: What RESPIN Actually Gives Us

RESPIN’s Marathi subset contains 4 dialect groups: D1 South Konkan (Sindhudurg/Ratnagiri), D2 North Konkan (Palghar/Thane), D3 Standard Pune, D4 Varhadi (Nagpur/Amravati).

Our primary targets are **Varhadi (D4) → Standard Pune (D3)** and **South Konkan (D1) → Standard Pune (D3)**: well-documented phonological contrasts, large speaker population, and RESPIN provides ~1,200h of annotated speech.

L1 Reference Ground Truth. For L1 ground truth, we use **IndicVoices-R Marathi** (studio-quality standard Marathi) + OpenSLR 103 alongside RESPIN D3 (Standard Pune). IndicVoices-R Marathi is the L1 reference for the TTS backbone and evaluation.

Three-Experiment Plan

1. **Primary: Varhadi → Standard Pune** (RESPIN Marathi D4 → D3 / IndicVoices-R Marathi). Well-documented phonological contrasts (retroflex lateral lenition, case marking), large dataset, clear primary target.
2. **Secondary: South Konkan → Standard Pune** (RESPIN Marathi D1 → D3). Coastal dialect variation, establishing direct baseline for Konkani speech adaptation.
3. **Extended: Hindi Awadhi Bundeli → Khariboli** (RESPIN Hindi D3 → D1 / IndicVoices-R Hindi). Proves language-agnosticism on Hindi across 5 RESPIN dialects (~2,500h).

This gives a multi-dialect, multi-language paper: *one pipeline, one dataset, three dialect pairs, two languages*.

All the Speech Data We Can Use

L1 Data (Standard Speech)

Dataset	Hours	Notes
IndicVoices-R Marathi https://huggingface.co/datasets/ai4bharat/indicvoices_r	100+	22 langs, primary Marathi L1 reference source
OpenSLR 103 Marathi https://openslr.org/103/	99	3 sociolect groups, standard Pune audio
OpenSLR 64 Marathi https://openslr.org/64/	28	Google Marathi read speech
IndicVoices-R Hindi https://huggingface.co/datasets/ai4bharat/indicvoices_r	175+	Secondary L1 source for extended Hindi experiments

L2 Data (Dialect Speech)

Marathi (primary target).

- **RESPIN-S1.0:** <https://spiredatasets.ee.iisc.ac.in/respincorpus> (https://github.com/labspire/respin_baselines.git) — ~1,200h Marathi, 4 dialects (Konkan South, Konkan North, Standard Pune, Varhadi). Direct download. **Primary L2 training source.**
- **OpenSLR 103:** <https://openslr.org/103/> — 99h, 31 speakers, 3 sociolect groups (college students, rural low-income, urban low-income). Direct download.
- **LDC-IL Marathi:** <https://data.ldcil.org/marathi-raw-speech-corpus> — 89h, 307 speakers, 4 regions (Marathwada, Puneri, Vidarbha, Goa). Paid.

Hindi (secondary target).

- **RESPIN-S1.0:** <https://spiredatasets.ee.iisc.ac.in/respincorpus> (https://github.com/labspire/respin_baselines.git) — ~2,500h Hindi, 5 dialect groups (Khariboli, Awadhi/Bundeli, Braj/Kanauji, Marwari, Garhwali). Direct download, clean/noisy splits.
- **LAHAJA:** <https://huggingface.co/datasets/ai4bharat/Lahaja> — 12.5h, 132 speakers, 83 districts. Test-split evaluation benchmark for Hindi.

Cross-Lingual Data (Method Validation)

- **Svarah:** <https://huggingface.co/datasets/ai4bharat/Svarah> — 9.6h Indian-accented English, 117 spk, 65 districts, 19 L1s.
- **L2-ARCTIC:** <https://psi.engr.tamu.edu/l2-arctic-corpus/> — Hindi-accented English (~2h) with manual mispronunciation tags.
- **AccentDB:** <https://accentdb.github.io/> — Parallel accented English with 4 Indian L1s (Hindi, Tamil, Telugu, Marathi).

- **VAANI:** <https://huggingface.co/datakers/vaani> — 31K+ hours, 165 districts, 109 languages. District-level dialect proxy.
- **RASMALAI:** <https://huggingface.co/datasets/ai4bharat/RASMALAI> — 13K hours, 23 languages, 24M dialect/emotion/style annotations. Powers IndicParlerTTS.

What’s Missing

- **No L1–L2 parallel pairs** for any Indic language — nobody has recorded the same speaker in both dialect and standard.
- **No cross-dialect parallel speech** — L2 datasets are standalone, not aligned to L1.
- **No standard DN evaluation benchmark** for Indic languages (LAHAJA is close but test-only).
- **Spontaneous dialect speech** — RESPIN is read speech; real-world dialect speech is more variable.

Data Strategy: What’s Realistic on a 32GB GPU

You asked whether individual datasets are sufficient or if you need to merge. Short answer: **one dataset can work, but you will merge two.** Here is the practical breakdown.

What Each Dataset Alone Gets You

Dataset	Hours	Dialect Labels?	Free?	Alone, Is It Enough?
RESPIN-S1.0 Hindi	~2,500	5 dialect groups	Yes (GitHub)	Yes — primary L2 training source
LAHAJA	12.5	83 districts	Yes (HF)	No — test-only, use for eval + text source
IndicVoices	23.7K	District metadata	Yes (HF)	L1 TTS only (no explicit dialect labels)
IndicVoices-R	175 Hindi	No dialect	Yes (HF)	L1 TTS backbone
VAANI	31K+	165 districts	Yes (HF)	Supplementary dialect proxy
Nirantar	3,240	208 districts	Form-based	Supplementary extempore data
RESPIN-S1.0 Marathi	~1,200	4 dialects	Yes (GitHub)	Yes — primary Marathi L2
OpenSLR 103 Marathi	99	3 groups	Yes	Yes — Marathi L2 supplement

The Minimum Viable Prototype (MVP)

You do not need all datasets. You need exactly:

1. **One voice-cloning TTS backbone (for L2 synthesis).** **IndicF5 (AI4Bharat)**, a reference-audio-cloning model (0.4B, flow-matching; fits 32GB; Marathi + Hindi; 10+ community fine-tunes) trained/fine-tuned on IndicVoices-R Marathi (100h) + RESPIN Marathi dialect audio. It clones the L1 speaker’s voice from a reference clip and applies dialect pronunciation learned via fine-tuning.

2. **One L2 training source.** RESPIN-S1.0 Marathi (~1,200h, 4 dialect groups: Konkan South, Konkan North, Standard Pune, Varhadi) + OpenSLR 103 (99h). Correct dataset at <https://spiredatasets.ee.iisc.ac.in/respincorpus>. Explicit dialect labels, train/dev/test splits. Primary training data for the VQ tokenizer and AR converter.
3. **One L1 target source (real, not synthetic).** IndicVoices-R Marathi / OpenSLR 103 real recordings (~5,000 utterances) paired with their transcripts. Authentic L1 side of every training pair — per CosyAccent, the L1 target must be real audio to avoid TTS artifacts.
4. **One evaluation benchmark.** RESPIN Marathi D4/D1 test audio paired with standard Pune transcripts. For each utterance: transcribe raw L2 audio with IndicConformer / IndicWhisper → compute WER against standard text. After conversion, measure WER again. The WER reduction is our primary metric.

That is it. Four data sources, all freely downloadable, all with dialect annotations.

Why this combination works. RESPIN-S1.0 gives us dialect-labelled L2 training data (4 Marathi dialects, 5 Hindi dialects) with direct download from <https://spiredatasets.ee.iisc.ac.in/respincorpus>. IndicVoices-R Marathi + OpenSLR 103 provide the real L1 target audio. **Note:** Our primary Marathi pair is Varhadi (RESPIN D4) / South Konkan (RESPIN D1) → Standard Pune (D3). **Also note:** the L1 side must be real audio (not TTS synthesis), per CosyAccent.

Do you need RESPIN for MVP? Yes — it is the primary freely available source of large-scale dialect-labelled Marathi speech.

32GB GPU Budget

Your single GPU is the main constraint. Here is what fits:

Component	Approach	GPU Memory
L1 TTS backbone	Fine-tune IndicF5 (0.4B, reference-audio cloning) on IndicVoices-R Marathi + RESPIN dialect	~12 GB
Dialect-conditioned L2 synthesis	IndicF5: X's voice clip as reference + dialect LoRA (Path 1) or separate accent embedding (Path 2)	Same model, no extra
VQ tokenizer	Small codebook (512 codes, 256 dim)	~4 GB
AR converter	6-layer transformer, 8 heads, 512 hidden	~6 GB
ECAPA-TDNN speaker encoder	Pretrained, frozen during VQ training	~2 GB
Flow-matching decoder	Skipped for MVP, use Griffin for waveform	~0 GB
Dialect classifier (eval)	Small CNN on HuBERT features	~4 GB
GRPO RL fine-tuning	LoRA adapters, batch size 4	~8 GB

Everything fits in 32GB with room for activations. The bottleneck is the AR converter training on synthetic pairs (sequence length × batch size). Use gradient accumulation and bfloat16.

What We Are Not Doing in the Prototype

- **Multilingual extension (Marathi + Hindi).** That adds data and model size. Start with Marathi only.

- **Full RESPIN 10K pipeline.** Use the Marathi/Hindi subsets directly instead.
- **Flow-matching decoder.** Griffin or HiFi-GAN is good enough for the prototype. Flow-matching adds 2–4 GB.
- **UTMOS training.** Use a pretrained UTMOS model for filtering; no GPU cost.

Step-by-Step Prototype Plan

1. **Download:** IndicVoices-R Marathi (HF, accept terms), OpenSLR 103 (direct), RESPIN-S1.0 Marathi (GitHub, direct download).
2. **Fine-tune IndicF5** on IndicVoices-R Marathi + RESPIN dialect audio to improve Marathi output quality, cloning fidelity, and dialect pronunciation.
3. **Prepare L1 targets (real audio):** Use real standard-Marathi recordings from IndicVoices-R Marathi / OpenSLR 103 (~5,000 utterances) paired with their transcripts.
4. **Synthesise Task 1 L2 pairs:** For each (real-L1-audio, text) pair, using a voice-cloning TTS backbone conditioned on the L1 speaker’s voice:
 - Generate **L2 audio** (same text) in the L1 speaker’s cloned voice with prompt “Varhadi accent”
 - Quality-filter (ASR consistency vs L1 transcript, speaker similarity to real L1, UTMOS)

Result: ~3,200–5,000 (real-L1, synthetic-L2) parallel pairs.
5. **Train VQ tokenizer:** Train on combined synthetic + real audio (RESPIN Marathi + OpenSLR 103). 512-codebook, 256-dim, 4 GB budget.
6. **Train AR converter:** 6-layer transformer, L2 tokens → L1 tokens. Train on synthetic pairs.
7. **Evaluate on RESPIN Marathi Test:** Take real Varhadi/South Konkan audio → convert through model → transcribe with IndicConformer/IndicWhisper → WER against standard text. Compare with raw L2 WER (“before”).
8. **Optional:** GRPO LoRA fine-tuning with WER + dialect classifier reward.

Expected timeline: 2–4 weeks for a working prototype. First results visible after step 6 (AR converter).

Merging Datasets: When and Why

Scenario	Datasets to Merge	Why Merge
MVP (Marathi Varhadi → Standard)	RESPIN Marathi D4 subset + OpenSLR 103	RESPIN provides Varhadi L2 training data;
Add South Konkan dialect	+ RESPIN Marathi D1 subset	OpenSLR/IndicVoices-R provides L1 reference
Add Hindi Awadhi	+ RESPIN Hindi D3 subset + IndicVoices-R	Coastal dialect variation baseline for Konkani adaptation
Scale up (10K pairs → 100K)	+ LDC-IL Marathi + VAANI subsets	For cross-lingual generalisation on Hindi Awadhi → Khariboli
		More utterances for synthetic generation

The prototype needs **RESPIN Marathi + OpenSLR 103 / IndicVoices-R Marathi**. Everything else is additive.

Phonological Contrasts (What Actually Changes)

These are the concrete sound changes the model needs to learn.

Marathi: Varhadi → Standard Pune (Primary Experiment)

Pattern	Varhadi (L2)	Standard (L1)
Retroflex lateral	[ɟ]	[ɭ]
Case marker	-le	-lā / -nā
Ergative marking	All persons	3rd person only
Dental/Palatal affricates	De-palatalised	Distinct palatal series
Nasalisation	Medial lenition	Preserved

Marathi: South Konkani → Standard Pune (Secondary Experiment)

Pattern	South Konkani (L2)	Standard (L1)
Schwa deletion	Reduced schwa syncope	Standard word-final schwa
Vowel raising	/o/ → [u], /e/ → [i]	Standard mid vowels
Affix variation	Coastal case endings	Pune postpositions
Lexical alignment	High Konkani lexical affinity	Standard Marathi vocabulary

Hindi: Awadhi Bundeli → Khariboli (Extended Experiment)

Pattern	Awadhi (L2)	Standard (L1)
Dative postposition	/ka:/	/ko:/
Locative marker	/ma:/	/mē:/
Affix variation	/-is/	/-ke/
Aspirated affricates	Merger with aspirated stops	Distinct
Retroflex stops	Less prominent	Full series

Pipeline design: Two-task cascade. Task 1: real L1 (IndicVoices-R Marathi / OpenSLR 103) + RESPIN text, passed through a voice-cloning TTS, gives synthetic L2 paired with real L1 (same speaker voice). Task 2: VQ tokenizer trained on all audio → AR converter trained on pairs → evaluate on real dialect audio. Primary pair: Varhadi → Standard Pune. See the “Two-Task Breakdown” and “Synthetic Parallel Data” sections.

Synthetic Parallel Data: The Source-Synthesis Pipeline

What problem are we actually solving?

To train our dialect-normalisation model, we need **parallel pairs**: for one sentence, we want two audio clips of the *same person* saying the *same words* — once in standard Hindi (the “L1” side) and once in a dialect

like Awadhi (the “L2” side). The model learns by looking at these pairs and figuring out “when the input sounds dialectal but the voice is X’s, output the same voice speaking standard.”

Here is the catch: **nobody has ever recorded this**. No dataset contains the same speaker saying the same sentence in both standard and dialect. Recording it from scratch would mean hiring bilingual dialect speakers, booking studio time, and collecting thousands of hours of audio. That is far too expensive and slow for a 12-week project.

So instead of recording it, we **make** the missing side with a computer. This is the single most important design decision in the whole project, and the rest of this section explains, step by step, how we do it without breaking the “same person, same words” requirement.

The core idea (borrowed from CosyAccent)

CosyAccent [3] (a 2026 paper) solved exactly this for English. Their trick, in plain words:

- We already *have* real standard speech — recordings of people speaking correctly. We use those as the L1 (standard) side. We do **not** synthesise the standard side.
- We **synthesise** (generate with a TTS — text-to-speech model) only the dialect side, but we make the TTS speak it **in the same person’s voice** as the real standard recording.

Why keep the standard side real instead of also generating it? A TTS never sounds perfectly natural — it leaves small robotic traces (“artifacts”). If we generated the standard side too, the model we train would learn to copy those robotic traces and think they are part of “standard speech.” That would make our output sound synthetic. By keeping the standard side as real human audio, we give the model a clean target to aim for. The paper calls this the key reason their method works: if you synthesise the target, “any TTS artifacts are inherited by the accent-normalisation model.”

How CosyAccent actually builds one pair

Imagine a speaker we’ll call **X**. CosyAccent does this:

1. **The L1 (standard) audio is real.** They take a genuine recording of X speaking in a standard accent. No TTS involved.
2. **The L2 (dialect) audio is synthesised in X’s own voice.** They use a voice-cloning TTS. A voice-cloning TTS is a model that, given a short sample of someone’s voice plus some text, reads that text in that person’s voice. CosyAccent feeds it X’s voice sample and the *same sentence*, but tells it to speak with a dialect accent. Out comes X’s voice — saying the dialect version of the sentence.
3. **Result: a clean parallel pair.** X (real) saying it standard, and X (synthetic) saying it in dialect. Same text, same speaker, only the accent changed.

That last point is the whole game: the pair differs *only* in dialect, not in who is talking. This is the **opposite** of generating both sides from one generic TTS voice, which would give you two different imaginary speakers and teach the model the wrong lesson.

CosyAccent result (to verify): The paper reports that its synthetic-data strategy performs comparably to real-parallel training. *The specific “0.3% WER gap” figure cited in earlier drafts has NOT been confirmed in the CosyAccent paper (arXiv:2602.19166) — treat it as unconfirmed and re-check against*

the primary source before citing.

Doing this for Indian languages

We follow the same two-side recipe, but we have something CosyAccent didn't lean on as heavily: **explicit dialect labels**. Datasets like RESPIN and RASMALAI tell us exactly which dialect a speaker uses (Awadhi, Marwari, ...), not just a vague “non-standard” tag. That lets us build specific, named pairs like Awadhi→Khariboli rather than a generic “dialect→standard.”

The one rule we must never break (straight from CosyAccent): the L1 side is **real** audio; only the L2 side is synthesised; and both sides share the **same speaker voice**.

Before we pick the TTS to do the synthesising, there is an important decision to get right. The box below explains it in full — read it before writing any code.

DECISION TO MAKE BEFORE WRITING ANY CODE (the “design fork”).

What we are trying to create. Given a real recording of Speaker X in standard Hindi, we want to create another recording that is:

- the **same speaker** (X),
- saying the **same sentence**,
- but with an **Awadhi accent**.

Only the accent should change.

The core problem. IndicF5 takes a *single* reference clip. That one clip contains **speaker identity** + **accent** + **speaking style** + **prosody** all mixed together. The model receives all of these as one bundle — it does **not** know which part is “voice” and which part is “accent.”

Two kinds of TTS.

- **Description-controlled TTS.** Example prompt: “A male speaker with an Awadhi accent.” The model *invents a new speaker* — it cannot preserve Speaker X. (This is IndicParlerTTS; fallback only.)
- **Voice-cloning TTS.** You provide a recording of Speaker X; the model tries to generate speech in X's voice. This is what IndicF5 does, and it is what we need.

The problem with IndicF5. The reference clip controls **both** who is speaking *and* how they speak. You **cannot** independently say “Voice = Speaker X, Accent = Awadhi,” because both come from the same reference clip.

Path 1 — LoRA (do not modify IndicF5). Train a LoRA on lots of Awadhi speech. The hope is that the LoRA learns *pronunciation patterns*, not specific speakers. At inference:

- Reference clip says: “sound like Speaker X”
- LoRA says: “pronounce words like Awadhi”

If both effects stay separate, the output is **Speaker X with Awadhi pronunciation**. If instead the LoRA also changes speaker identity, the output starts sounding like one of the RESPIN training speakers — then Path 1 fails.

The Week-1 experiment is simply: attach the Awadhi LoRA, give Speaker X as the reference, listen, and ask three questions: (1) Does it still sound like Speaker X? (2) Does it sound Awadhi? (3) Did both happen at the same time? If yes, Path 1 succeeds.

Path 2 — separate speaker and accent (modify IndicF5). Instead of *hoping* the model separates them, redesign it.

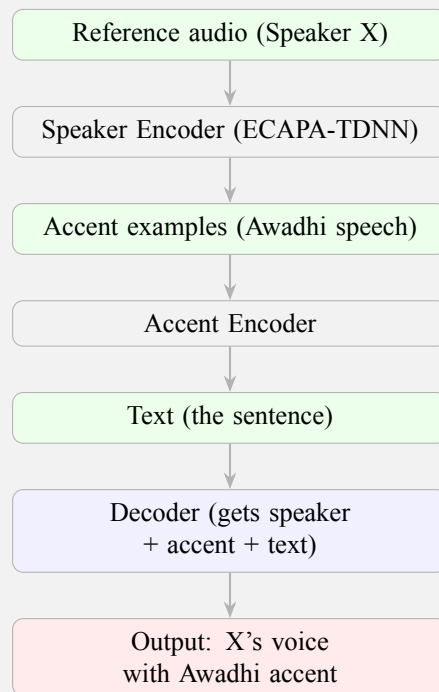
Current: Reference audio → one embedding → decoder.

Desired:

- Reference audio → Speaker Encoder → Speaker embedding
- Accent examples → Accent Encoder → Accent embedding
- Speaker embedding + Accent embedding + Text → Decoder

Now you **explicitly tell** the model: Speaker = X, Accent = Awadhi — instead of asking it to infer both from one clip.

Training. A Speaker encoder (e.g. ECAPA-TDNN) produces a vector for identity; an Accent encoder produces a vector for accent. The decoder receives all three (speaker vector + accent vector + text). Extra training losses encourage speaker preservation, correct accent, and good speech quality. This is conceptually like NVIDIA **VANI**, which uses separate conditioning for speaker, language, and accent.



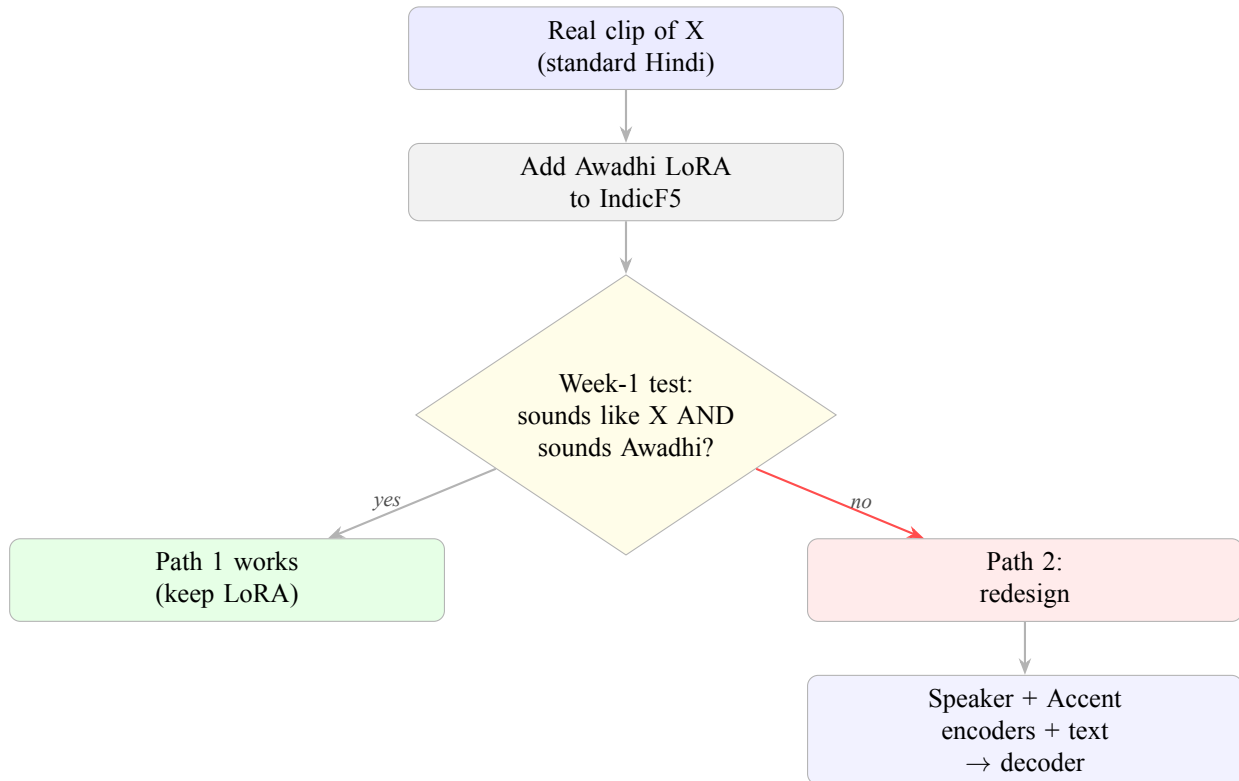
Main takeaway.

- **Path 1:** do not modify IndicF5; add a LoRA; hope the LoRA changes only pronunciation. Fast and easy. May fail.
- **Path 2:** modify IndicF5; add explicit speaker and accent conditioning. More engineering work. More reliable.

The entire fork boils down to one question: **Can a LoRA teach IndicF5 “how to pronounce” without changing “whose voice” it is copying?**

- If **yes** → use Path 1.
- If **no** → redesign the model (Path 2).

First move (30 min): check IndicF5’s 10+ community fine-tunes for an existing accent-conditioned variant. If none, run the Week-1 experiment above before committing.

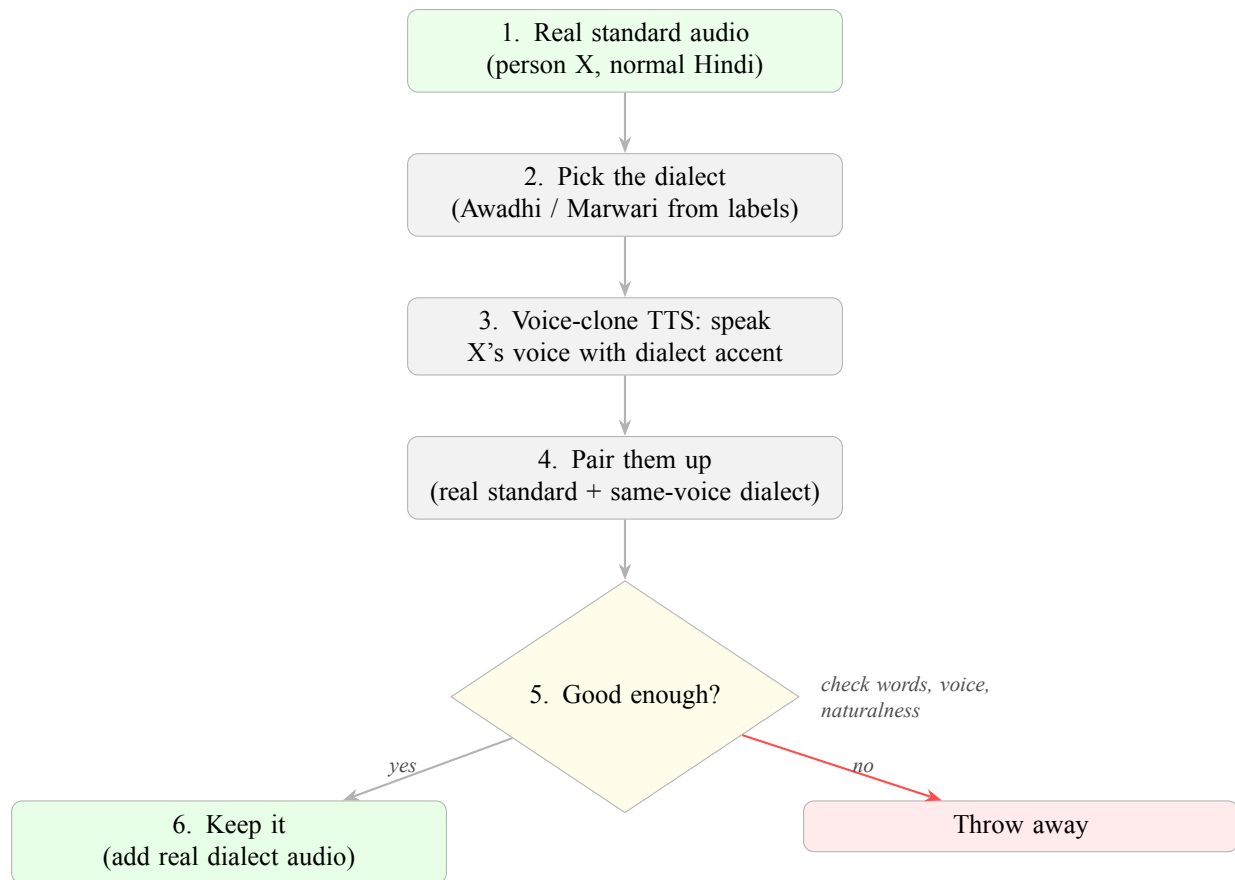


Building one training pair, step by step

Concretely, here is how we manufacture a single (real-L1, synthetic-L2) pair. The boxed fork above decides *which* TTS we use; the steps below are what we do with it.

1. **L1 target = real audio (not synthetic).** Use real standard-Hindi recordings from IndicVoices-R Hindi (175+ hours, CC BY 4.0) as the authentic L1 side of every training pair. **Do not synthesise the L1 side** (synthesising it injects TTS artifacts, which is exactly what CosyAccent avoids).
2. **Obtain L2 dialect references.** RASMALAI provides explicit dialect labels (“Awadhi-accented Hindi”, “Marwari-accented Hindi”, etc.) for 13K hours of speech across 23 languages. We extract per-dialect prototypes by averaging ECAPA-TDNN embeddings from known L2 speakers in the same dialect group (Awadhi speakers from RESPIN D3, Marwari speakers from RESPIN D4).
3. **Synthesise L2-dialectal speech in the L1 speaker’s cloned voice.** Per CosyAccent, the dialect is applied while the voice is cloned from the corresponding L1 speaker:

- **Voice-cloning backbone (preferred):** Use **IndicF5** (reference-audio cloning) fine-tuned on RESPIN dialect audio, passing **X’s real Khariboli clip as the voice reference** so output is in X’s voice. *Note IndicF5’s single-reference limitation:* the one reference clip controls both voice and prosody, so test Path 1 (LoRA accent leak while timbre anchored) in Week 1, or pursue Path 2 (separate accent conditioning, VANI-style) if timbre drifts.
 - **Description-based fallback:** IndicParlerTTS with RASMALAI descriptions (“A male speaker with an Awadhi accent”) + ECAPA-TDNN re-conditioning at decode. Accept weaker, unverified voice match.
4. **Generate the parallel pair.** For each (text, real-L1-audio) from the L1 set:
- Keep the **real L1 audio** as the standard target
 - Synthesise the **L2 audio** in the L1 speaker’s cloned voice with an Awadhi accent
- Result: (L1-audio, L2-audio) where L2 is the same sentence in an Awadhi accent, **spoken in the same voice as the L1 recording**.
5. **Quality filter.** Run each synthetic pair through:
- **ASR confidence check:** Transcribe both L1 and L2 with IndicWhisper. If WER between the two transcriptions exceeds 15%, discard (the dialect is so strong that content differs).
 - **Speaker similarity:** ECAPA-TDNN cosine similarity between L1 (real) and L2 (synthetic) audio. Discard if < 0.5 (voices diverged too much).
 - **Naturalness:** UTMOS score. Discard if < 3.0 .
6. **Augment with real L2 speech.** We also include real L2 recordings from LAHAJA and RESPIN as unpaired data. The VQ tokenizer sees both real and synthetic L2 speech, improving robustness.



Expected Yield

Stage	Prototype Count
LAHAJA normalised sentences (text source for synthesis)	~6,100
Synthetic pair generation attempts	~6,100
After ASR consistency filter (L1 and L2 transcribe to same text)	~4,800
After speaker similarity filter (ECAPA cosine > 0.5)	~4,000
After UTMOS filter (score > 3.0)	~3,200
Final usable parallel pairs for MVP	~3,200
Real L2 utterances for evaluation (LAHAJA audio)	~6,100
Real L1 utterances for backbone fine-tuning (IndicVoices-R Hindi)	~5,000

3,200 pairs is enough to train a proof-of-concept AR converter. For reference, TokAN's original training used ~10,000 pairs per dialect. If 3,200 shows a WER reduction, scaling up is straightforward — the synthesis pipeline is automated, not manual.

Final scale target (with RESPIN): ~100,000 pairs after all filters. But do not start there. Prove it works with 3,200.

Comparison: Why This Beats Alternative Data Strategies

Strategy	How It Works	Problems
Human-recorded pairs	Record same speaker in L1 and L2	Impractical at scale; no parallel data exists
Unpaired cycle-consistency	CycleGAN/VQ-VAE loop without parallel pairs [17]	Lower quality; no direct supervision signal
Metric-based (no pairs)	Direct dialect classification loss	Hard to optimise; no paired reconstruction
Our approach (CosyAccent-style)	Synthetic L2 + real L1 parallel pairs, voice-cloned	Effective per CosyAccent (gap to real pairs to be verified against paper)

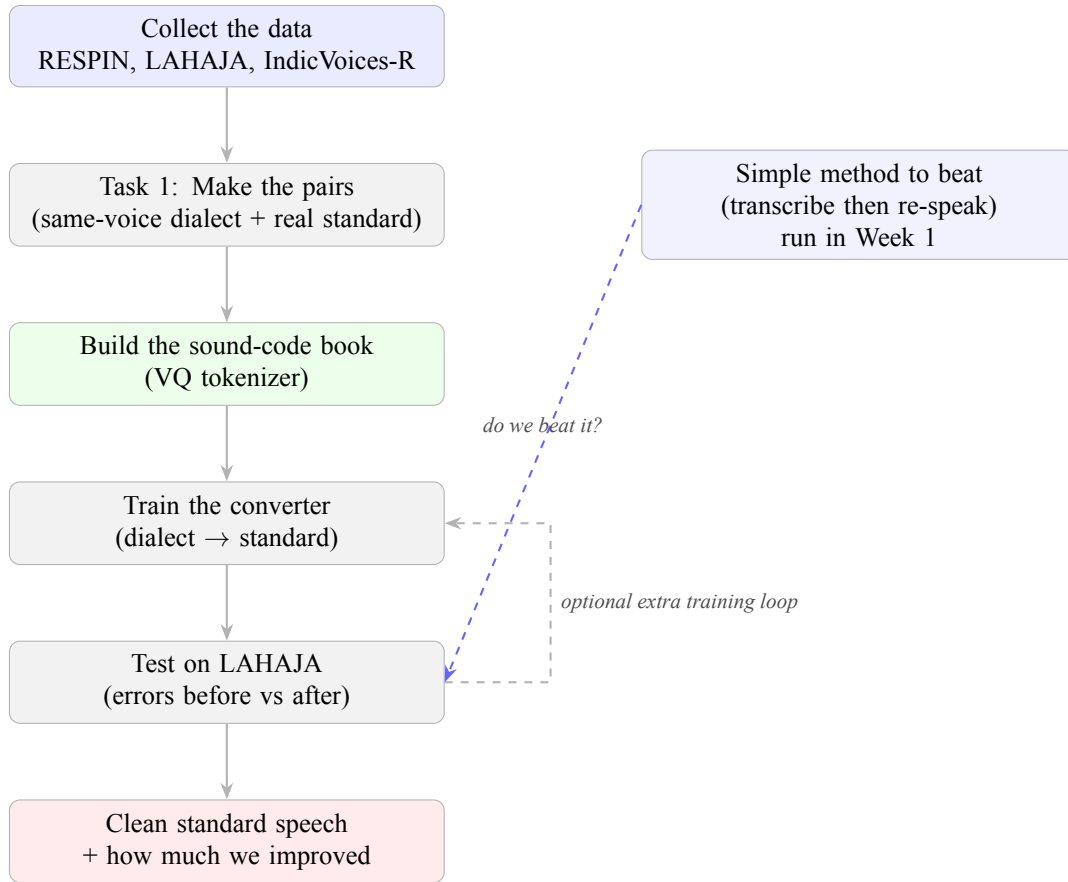
Key Difference from CosyAccent: Dialect-Labelled Conditioning

CosyAccent uses speaker embeddings as dialect proxies — no explicit dialect labels needed. But we *have* dialect labels (LAHAJA districts, RESPIN dialect groups, RASMALAI descriptions). This lets us:

- **Synthesise specific dialect pairs:** Awadhi → Khariboli, Marwari → Khariboli, Varhadi → Pune, etc., not just “non-standard → standard”
- **Control dialect strength:** interpolate between pure Awadhi and standard Hindi embeddings
- **Multi-dialect training:** train one model on all Hindi dialects simultaneously, sharing the VQ codebook
- **Extend to new dialects:** given a new dialect’s prototype, synthesise pairs without re-recording

Two-Task Breakdown: Data vs Model

The project splits into two sequential tasks:



Task 1: L1-L2 Synthetic Pair Generation (Data Pipeline)

Create (L1_audio, L2_audio) parallel pairs for training the normalisation model.

Inputs:

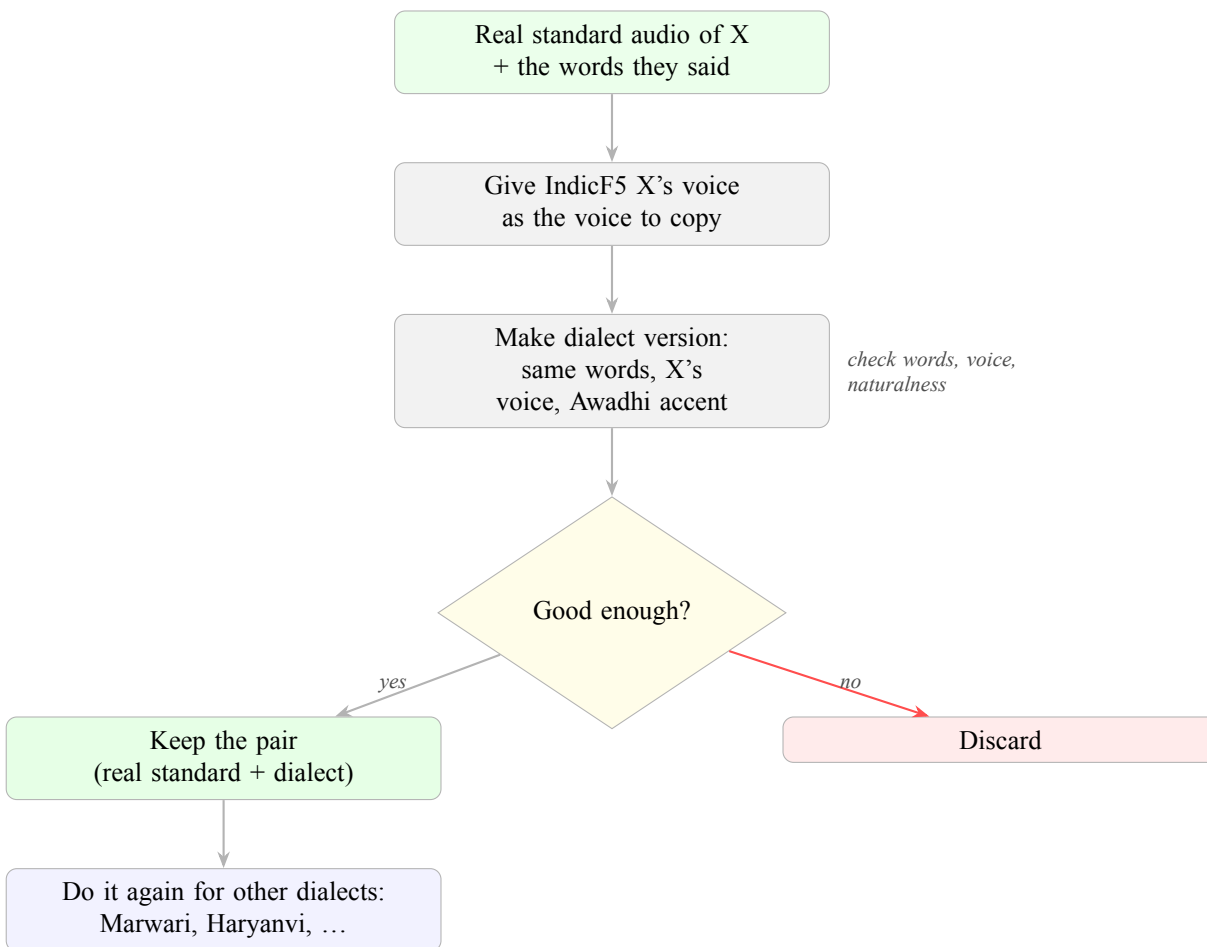
- **L1 target audio:** Real standard-Hindi recordings from IndicVoices-R Hindi (~5,000 utterances). **These are real, not synthesised** (per CosyAccent: synthesising the L1 target injects TTS artifacts).
- **Text source:** The text corresponding to each L1 utterance.
- **TTS backbone: IndicF5** (reference-audio cloning) fine-tuned on RESPIN dialect audio, driven by X's real voice sample as the identity reference. Pass X's clean Khariboli clip as reference so L2 output is in X's voice with Awadhi pronunciation (test Path 1 timbre-anchoring in Week 1). IndicParlerTTS (description-based, no cloning) is a fallback relying on ECAPA-TDNN re-conditioning at decode (see design fork).

Process:

1. Take a real L1 utterance (speaker X, standard Hindi).
2. Synthesise the L2 audio for the *same text* using the TTS backbone conditioned on speaker X's voice reference and the target dialect ("Awadhi accent").
3. Optionally vary accent control for multi-dialect training: Marwari, Haryanvi, etc., each cloned into speaker X's voice.

4. Quality-filter pairs (ASR text consistency, ECAPA speaker similarity to the real L1, UTMOS naturalness).

Output: ~3,200–5,000 (real-L1, synthetic-L2) parallel pairs where L2 is the same sentence in a dialect accent, spoken in the *same voice* as the real L1. Speaker identity is matched by construction (voice cloning), not assumed from dialect-conditioned TTS.



Task 2: Normalisation Model Training & Evaluation

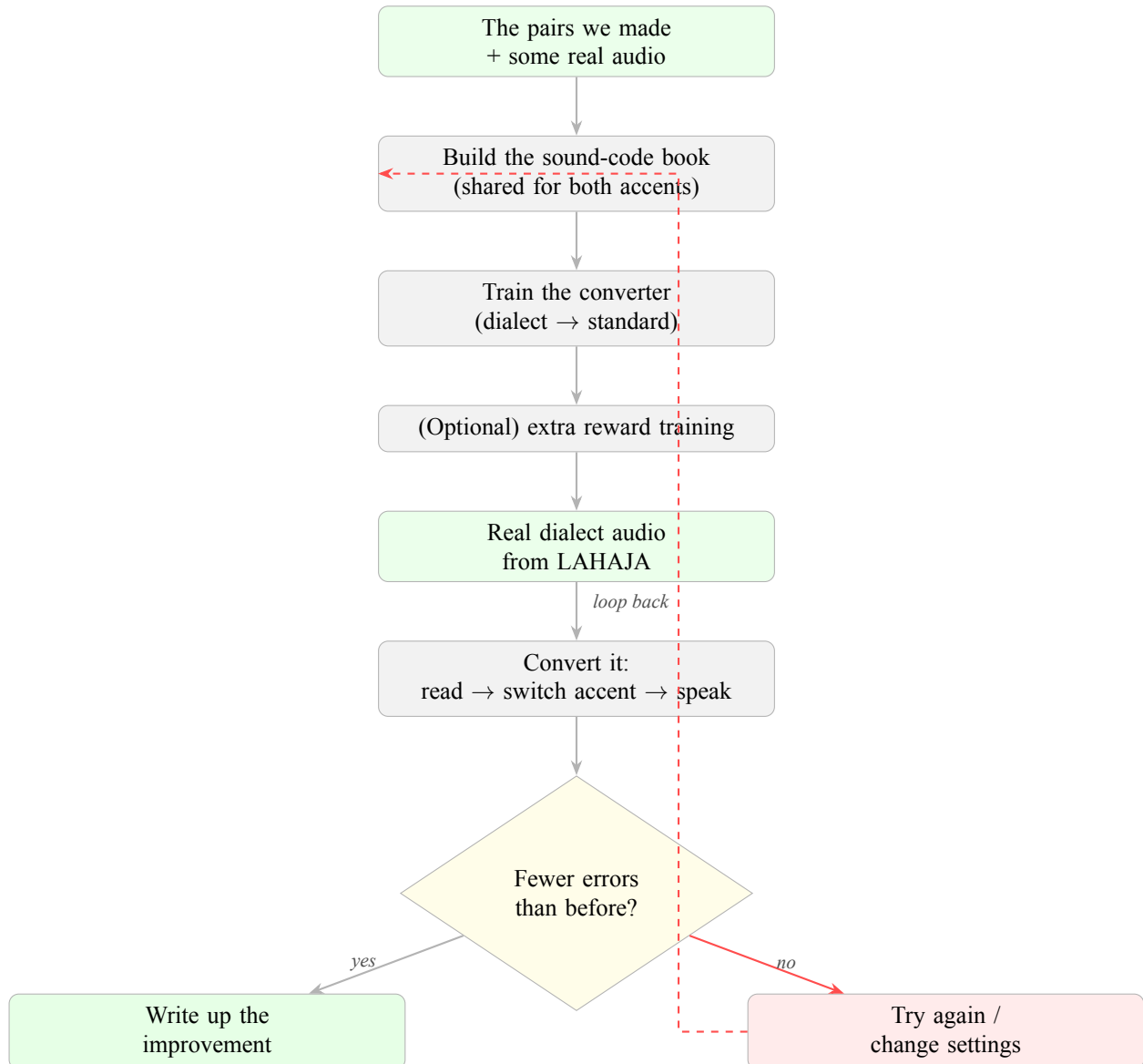
Training:

- **VQ tokenizer:** Trained on all available audio — synthetic L1, synthetic L2, real L1 (IndicVoices-R), real L2 (LAHAJA). Shared codebook ensures L1 and L2 map to the same acoustic tokens.
- **AR converter:** Trained on synthetic (L2 → L1) token sequences from Task 1. 6-layer transformer, 512 hidden, 8 heads.
- **Optional:** GRPO RL fine-tuning with WER + dialect classifier reward.

Evaluation: Use LAHAJA as a clean benchmark. For each of ~6,100 LAHAJA utterances (real L2 audio with known L1 transcript):

1. Compute **before WER**: transcribe raw LAHAJA audio with IndicWhisper, measure WER against LAHAJA normalised text.

2. Run audio through trained normalisation model (VQ encode → AR convert → waveform decode).
3. Compute **after WER**: transcribe converted audio with same ASR, measure WER against same normalised text.
4. Target: relative WER reduction > 30% (e.g., 25% → 17.5%).



Why This Separation Matters

- **Task 1 is fully automated.** Once the TTS backbone is ready, generating 5K–100K pairs is a batch job. Scale is cheap.
- **Task 2 is the research contribution.** The VQ-AR design, dialect conditioning, GRPO fine-tuning, and evaluation protocol are where novelty lives.
- **Task 1 re-uses existing tools.** Voice-cloning TTS + LAHAJA/RESPIN transcripts + real IndicVoices-R L1 = synthetic pairs. No new parallel data collection needed.

- **Task 2 is language-agnostic.** The same VQ-AR pipeline works for Hindi, Marathi, etc. Only the TTS backbone changes per language.

Summary Table

	Task 1 (Data)	Task 2 (Model)
Goal	Create (L1, L2) pairs	Train normalisation model
Input	LAHAJA normalised text	Synthetic pairs + real audio
Tool	Voice-cloning TTS + filtering	VQ tokenizer + AR converter
Output	~3,200 synthetic pairs	WER-reduced L1 speech
Eval	ASR/UTMOS/speaker sim filters	LAHAJA WER benchmark
Scale cost	Batch job (GPU hours)	Training (GPU days)

Multilingual Normalisation Under a Single Model

A key question: can one model handle dialect normalisation across *multiple* Indic languages? This is under-explored in the literature, but several recent papers point to how it could work.

Why It Matters

Building separate models for Hindi, Marathi, etc., is expensive and doesn't scale. A single multilingual model would:

- Share data across languages — Hindi data helps Marathi and vice versa
- Handle code-mixed speech naturally (Hindi–English, Marathi–English)
- Serve as a single deployment for all Indic languages
- Transfer well to low-resource languages that have little to no L2 data

Architecture Approaches from Recent Work

1. Shared codebook with language ID conditioning (closest to TokAN). TokAN already uses a VQ tokenizer with shared codebooks across L1 and L2. Extending this to multiple languages means:

- A shared VQ codebook across all languages (Hindi, Marathi, etc.)
- Language ID or dialect ID embedding injected at the encoder or decoder
- The model learns which phonological mappings are language-specific and which generalise

This is the most straightforward extension. The VQ tokens become language-agnostic acoustic units. The decoder learns language-specific conversion via the language embedding. Related work: **MulliVC** [17] uses cycle-consistency for multilingual voice conversion with shared representations; **VALL-E X** uses language ID tokens in a neural codec language model for cross-lingual synthesis.

2. Disentangled dialect/speaker/content representations. **CrossAccent-TTS** (Sony Research India, INTERSPEECH 2026) [16] proposes an *Accent Intensity Controller (AIC)* that injects weighted language embeddings into a disentangled dialect subspace, enabling smooth interpolation between dialects. This directly applies to multilingual DN:

- Speaker, content, and dialect are modelled as separate subspaces
- At inference, the dialect subspace is set to “standard” regardless of input, while speaker subspace preserves identity
- The same architecture handles Hindi Awadhi → Khariboli and Marathi Varhadi → Pune — just change the dialect embedding
- Dialect intensity is controllable: you can dial how much normalisation to apply

Their experiments on Indic Multilingual and L2-ARCTIC datasets show that a single model with disentangled representations can handle multiple language pairs without per-language fine-tuning.

3. Language-agnostic phonetic tokens. **Accent Normalisation Using Self-Supervised Discrete Tokens** (INTERSPEECH 2025) [18] uses a self-supervised model (HuBERT/wav2vec2) to extract discrete tokens that capture phonetic content independent of accent or language. A conversion model then maps L2 tokens → L1 tokens. This approach:

- Naturally extends to multilingual because the SSL tokens are language-agnostic
- The conversion model is trained on all L2 → L1 pairs across languages
- Language ID can be used as an additional conditioning signal

4. Multilingual TTS backbone with dialect conditioning. **Optimized Multilingual TTS for Accents & Emotions** (2025) [21] trains a single TTS model with dialect and emotion embeddings across Indic languages. For DN, the pipeline becomes: transcribe L2 speech → re-synthesise in standard dialect using the TTS with standard dialect embedding. The key insight: a single TTS trained on multiple languages with dialect labels can serve as the synthesis backend for all of them.

Our Tentative Approach

Proposed architecture: TokAN-style VQ tokenizer + autoregressive converter. Primary experiment: Hindi Awadhi → Khariboli. Secondary: Marathi Varhadi → Pune. Extended: Hindi Marwari → Khariboli. Shared VQ codebook across L1 and L2. Speaker conditioning via ECAPA-TDNN embeddings injected into the flow-matching decoder. GRPO RL post-training with WER + dialect classifier rewards. See the “Two-Task Breakdown” section for the concrete data flow and model training plan.

Cross-Lingual Transfer: What Should Generalise

If the multilingual approach works, certain phonological patterns should transfer across languages:

Pattern	Appears In	Shared Across
Dative substitution	Awadhi, Varhadi	Hindi, Marathi
Locative marker change	Awadhi	Hindi
Case marker substitution	Awadhi, Varhadi	Hindi, Marathi
Ergative extension	Awadhi, Varhadi	Hindi, Marathi
Retroflex lateral	Varhadi	Marathi

If the model learns that “fricativisation of affricates” is a pattern that occurs in non-standard varieties of both Hindi and Marathi, it should be able to handle a new language-pair with similar phonology from minimal data.

Open Questions

- **How to handle different scripts?** Hindi (Devanagari) vs Marathi (Devanagari + Modi) — may need script-agnostic or IPA-based representations
- **How to evaluate cross-lingual transfer?** If we train on Hindi, does Marathi benefit from shared representations? Need controlled experiments
- **How much data per language?** What’s the minimum L2 data needed for a new language to work in the shared model?
- **Code-switching:** Hindi–English and Marathi–English code-mixing is ubiquitous — should the model also normalise English insertions to native-accented English?

Do We Even Need This?

Honest question. Dialect normalisation is an odd niche — even for English, only a handful of papers exist and no production system uses it. Before we build this, let’s examine whether the problem is real and whether our approach makes sense.

The Case Against

1. **ASR is getting better, not worse.** Whisper, IndicWhisper, and IndicConformer already handle dialectal speech reasonably well. A decade ago dialect-dependent WER gaps were 10–20%; today they’re 2–5%. Soon ASR might just work for everyone without dialect normalisation.
2. **TTS prompting already solves part of this.** If I want to hear text in a standard accent, I can already run Whisper → IndicParlerTTS. Transcribe the L2 speech to text, then re-synthesise in a standard voice. This “cascaded” approach works today for ASR-transcribable speech.
3. **Code-switching is more important.** Most Hindi speakers switch between Hindi and English mid-sentence. An Awadhi speaker code-switching between Awadhi and Hindi is even more common. Dialect normalisation that ignores code-switching solves a small part of the real problem.
4. **Speaker identity may not survive.** If we strip the dialect and keep the voice, what does “voice” even mean for a speaker whose L1 phonology is fundamentally different? An Awadhi speaker without Awadhi features might sound like a different person.

5. **Who benefits?** ASR improvements help everyone. Dialect normalisation mainly helps downstream consumers of dialectal speech (call centres, voice assistants). The speakers themselves don't benefit — they're already understood by humans.
6. **No benchmark, no community.** There is no test set, no leaderboard, no shared task for Indic dialect normalisation. We'd be building the evaluation and the model simultaneously, which makes it hard to measure real progress.

The Case For

1. **Cascaded (ASR → TTS) loses everything.** Re-synthesis discards prosody, emotion, emphasis, and speaker identity. The output sounds robotic and lacks the original speaker's expressiveness. Dialect normalisation preserves all of that — it only changes the phonological features that mark the dialect.
2. **ASR errors compound.** In a cascaded system, the TTS output is only as good as the ASR transcription. For heavy dialects (Awadhi, Chhattisgarhi), ASR can produce 30–40% WER. That means a third of the re-synthesised output is wrong content. End-to-end dialect normalisation avoids this intermediate transcription bottleneck.
3. **The ASR gap is real for “low” dialects.** For Chhattisgarhi, Magahi, and heavily Awadhi-accented Hindi, Whisper WER is >30% compared to <10% for Khariboli. This is not “soon solved” — these dialects have almost zero training data in ASR models. Dialect normalisation is a different approach: convert to standard phonology first, then run any standard ASR.
4. **Code-switching is addressable.** The multilingual section shows how a single model can handle Hindi and Marathi simultaneously. Code-switching (Hindi–English, Awadhi–Hindi) is a natural extension — the model learns normalisation for each language component separately.
5. **Speaker identity has been shown to work.** TokAN preserves speaker similarity (ECAPA cosine > 0.85) while reducing WER. The flow-matching decoder conditioned on the source speaker embedding is the key insight — it regenerates the waveform in the standard dialect but with the original speaker's voice qualities.
6. **Downstream is where the impact is.** Voice assistants (Amazon Alexa Hindi, Google Assistant Hindi) handle Khariboli well and everything else poorly. Call centre analytics for Awadhi-speaking regions fails. Dubbing and content localisation for OTT platforms (Netflix, Hotstar) can use dialect normalisation to make regional content accessible to standard-Hindi-speaking audiences while keeping the original speaker's identity.
7. **Novelty is the point.** No one has built this for any Indian language. The first-mover advantage for setting benchmarks, releasing datasets, and defining the evaluation protocol is significant. This is a publication pipeline — first paper on synthetic data strategy + primary results (conference), second on multi-dialect + ablations (journal), third on multilingual extension.

Bottom Line

Would I build this if someone else had already done it? Probably not — the ASR gap is closing. But *no one has done it*, and the gap for “low” dialects (30–40% WER) is still large enough to justify the effort. The project's value is as much in the data creation, benchmark definition, and architecture exploration as in the final model output. It is a research contribution, not a product.

The biggest risk is that cascaded ASR → TTS improves enough in the next 1–2 years to make end-to-end dialect normalisation irrelevant. But that would require (a) ASR that works at <10% WER on Awadhi, Chhattisgarhi, and Magahi, and (b) TTS that preserves speaker identity during re-synthesis. Neither is close today.

How We’ll Measure Success

Metrics

- **WER:** IndicWhisper or Whisper large-v3 fine-tuned on the target language — primary metric, measured before and after conversion
- **Relative WER reduction:** $(WER_{before} - WER_{after}) / WER_{before}$ — target >30% relative reduction
- **Dialect classifier score:** Train a CNN on HuBERT features from RESPIN dialect data. Report classification accuracy before and after conversion. Target: classify as “standard” with >80% accuracy after conversion.
- **Speaker similarity:** ECAPA-TDNN cosine similarity (input vs converted). Target: >0.85 (comparable to TokAN’s English results)
- **Quality:** UTMOS, DNSMOS. Target: no degradation from input (converted speech should not sound worse than input)
- **Per-dialect analysis:** Report all metrics separately for each dialect pair (Awadhi, Marwari, Garhwali → Khariboli; Varhadi → Pune)

Baselines

1. Raw L2 input (lower bound — no conversion)
2. Cascaded ASR → TTS (Whisper → IndicParlerTTS standard mode) — the most important baseline; if we don’t beat this on speaker similarity + WER, the end-to-end system is not justified
3. Frame-to-frame GAN conversion (standard voice conversion baseline)
4. Direct flow-matching on Mel spectrograms (without VQ tokenisation)
5. TokAN-style VQ tokenizer (scaled for target language, English-trained weights transferred)

Ablation Studies (Required for Journal)

Scope warning (reviewer note): The 7-ablation list below, plus per-phoneme error analysis and cross-dialect transfer, is roughly a journal paper’s worth of experiments for a single 32GB GPU in 12 weeks. Treat only what is needed for the *conference* paper as committed scope; move the rest to “if time permits”. Be realistic about what fits before locking the plan.

Every architectural component must be individually justified. The following ablations are planned:

Ablation	What It Tests	Expected Insight
VQ vs direct spectrogram	Whether discrete tokenisation helps continuous representations	VQ should improve by reducing acoustic variability
With/without GRPO RL	Whether RL fine-tuning adds value beyond SFT	GRPO should give 1–3% additional WER reduction
Synthetic-only vs synthetic+real	Whether real L2 data in VQ training matters	Mixed training should improve robustness to real dialect speech
Codebook size (256, 512, 1024)	Capacity vs efficiency trade-off	512 likely optimal for 32GB GPU; 1024 may overfit
AR converter depth (3, 6, 12 layers)	Model capacity for token conversion	6 layers likely sufficient; 12 may overfit on 3K pairs
With/without speaker conditioning	Whether ECAPA-TDNN preserves identity	Speaker conditioning critical for similarity scores
Dialect strength interpolation	Controllable normalisation intensity	Partial normalisation may outperform full for some dialects

Error Analysis (Required for Journal)

After obtaining results, a detailed linguistic error analysis is essential. This is where domain knowledge pays off and where journal reviewers expect depth.

- **Per-phoneme analysis:** Which phonological patterns does the model fix well? For Awadhi → Khariboli: /ka:/ → /ko:/ (dative), /ma:/ → /mẽ:/ (locative). For Varhadi → Pune: retroflex lateral, case markers. Report per-pattern accuracy.
- **Failure modes:** Which patterns does the model miss or hallucinate? Does it over-normalise (removing dialect features that should stay in code-mixed speech)? Does it under-normalise phonemes that are acoustically close?
- **Dialect strength effect:** How does partial normalisation (interpolated dialect embeddings) compare to full normalisation? Is there a sweet spot where WER drops but speaker similarity stays high?
- **Cross-dialect transfer:** When trained on Awadhi → Khariboli, does the model partially generalise to Marwari → Khariboli without retraining? This tests whether the VQ codebook captures language-level abstractions.
- **Comparison to cascaded baseline:** Where does end-to-end beat cascaded ASR→TTS, and where doesn't it? Speaker similarity should clearly favour end-to-end; WER may be comparable.

Benchmark as Contribution

Since no Indic DN benchmark exists, the LAHAJA-based evaluation protocol is itself a contribution:

- **Evaluation methodology:** WER-before/after using IndicWhisper on real LAHAJA L2 audio against normalised transcripts. Document every preprocessing step.

- **Dialect classifier:** Train a small CNN on HuBERT features from RESPIN dialect data. Report classification accuracy before and after conversion.
- **Released artifacts:** Evaluation scripts, data splits, pre-trained dialect classifier, and baseline results. Makes the benchmark citable even by people who don’t use the model.
- **Leaderboard potential:** If the benchmark gains traction, it becomes the standard evaluation for future Indic DN work.

Publication Strategy

The project produces **two independent submissions**, one per phase (see “Project Split: Two Publishable Tasks”).

Paper 1 (end of Month 4)

Submit to **INTERSPEECH 2027** or **ICASSP 2027** as a complete results/system paper: first Indic dialect-normalisation system, the synthetic-data pipeline, the LAHAJA benchmark, and the cascaded-baseline comparison. After acceptance, expand to **TASLP** or **CSL** with the deeper single-language ablations and error analysis.

Paper 2 (end of Month 8)

Submit to **TASLP** or **CSL** (or a second conference if the multilingual/analysis angle fits better): multi-dialect + multilingual results, cross-lingual transfer analysis, and the Path 1 vs Path 2 disentanglement study. Paper 2 reuses Paper 1’s pipeline, so its writing starts from finished, logged code.

Experiment Logging

Keep a detailed experiment log from day one. Journal reviewers will ask for ablations you didn’t plan. If you logged everything (hyperparameters, data splits, random seeds, training curves), you can reconstruct them. If you didn’t, you’re rerunning experiments for months.

Reproducibility protocol: Log every experiment with: (1) data split IDs, (2) random seeds, (3) hyperparameters, (4) training time, (5) final metrics. Store in a structured format (JSON/YAML), not just terminal output. Release all artifacts at submission time.

Project Split: Two Publishable Tasks

Instead of one 12-week push, the project is split into **two independent, self-contained papers**, each with a **4-month timeline**. Each paper stands on its own (its own contribution, experiments, and writing) so that even if the second slips, the first is a complete publication. The split follows the natural dependency: Paper 1 proves the core idea end-to-end on one language; Paper 2 scales it and studies what generalises.

Task A — Paper 1 (Months 1–4): Method + Single-Language Proof of Concept

Research question: Can we build the first dialect-normalisation system for an Indic language (Marathi), using synthetic parallel data and an end-to-end VQ-AR + GRPO pipeline, and beat the cascaded baseline on both WER and speaker similarity?

Scope (committed):

- Language: Marathi primary. Primary pair: Varhadi / South Konkan → Standard Pune (RESPIN D4/D1 → D3 / IndicVoices-R Marathi).
- Data pipeline: IndicF5 voice-cloning (resolve the design fork in Month 1) → real L1 + cloned-voice L2 pairs.
- Model: VQ tokenizer (shared codebook) + AR converter + optional GRPO RL.
- Benchmark: RESPIN Marathi / OpenSLR 103 WER-before/after + ECAPA speaker similarity + dialect-classifier accuracy.
- Baselines: raw L2, cascaded ASR→TTS, frame-to-frame GAN, direct flow-matching.
- Ablations needed for the paper: VQ vs direct spectrogram; with/without GRPO; synthetic-only vs synthetic+real; with/without speaker conditioning.
- Error analysis: per-phoneme patterns (retroflex lateral lenition [ɭ]→[j]), case markers *-le*→*-lā*, failure modes.

Contribution statements (Paper 1):

1. First dialect-normalisation system for any Indic language (Marathi, Varhadi→Standard Pune).
2. The synthetic parallel-data pipeline (real L1 + voice-cloned L2) and its published recipe.
3. The Marathi evaluation benchmark as a citable artifact.
4. Empirical proof that end-to-end beats cascaded ASR→TTS on speaker similarity.

Target venue: INTERSPEECH 2027 or ICASSP 2027 (system/results paper), expanded to TASLP/CSL later.

Task B — Paper 2 (Months 5–8): Scale-Up, Multilingual, and Analysis

Research question: Does the method generalise across dialects and languages, and what phonological knowledge transfers?

Scope (committed):

- Additional Marathi pairs: South Konkan → Standard Pune (RESPIN D1) and North Konkan (RESPIN D2).
- Second language: Hindi Awadhi Bundeli → Khariboli (RESPIN D3 → D1/IndicVoices-R) + Marwari (RESPIN D4).
- Multilingual model: shared VQ codebook + language/dialect-ID conditioning across Marathi and Hindi.
- If Path 1 failed in Paper 1, Paper 2 adopts Path 2 (VANI-style separate speaker/accent conditioning) and studies its effect.
- Deeper ablations: codebook size (256/512/1024), AR depth (3/6/12), dialect-strength interpolation.
- Linguistic error analysis across dialects/languages; comparison of transfer patterns.

Contribution statements (Paper 2):

1. Multilingual, multi-dialect dialect normalisation under one shared model.
2. Cross-dialect and cross-lingual transfer analysis (what generalises, what does not).
3. A controlled study of disentanglement (Path 1 vs Path 2) for identity preservation.
4. Extended benchmark across Marathi + Hindi dialects.

Target venue: TASLP or CSL (journal), or a second conference (e.g. ACL/EMNLP-style for multilingual speech) if the analysis angle is stronger.

Why this split works

- **De-risks publication:** Paper 1 is achievable in 4 months and is publishable alone; Paper 2 does not block it.
- **Clean dependency:** Paper 1 delivers the pipeline + proof; Paper 2 reuses it and adds scale/analysis, so the second paper’s writing starts from finished code.
- **Both are novel:** Paper 1 = first system + data method + benchmark; Paper 2 = multilingual transfer + disentanglement study. Neither is ”just more results.”
- **Timeline buffer:** 4 months each leaves slack for the design-fork resolution and the inevitable re-runs.

Limitations (Honest Assessment)

- **Read speech only.** RESPIN is read speech. Real-world dialect speech is spontaneous, code-mixed, and more variable. The model’s performance on spontaneous speech is unknown until tested.
- **No code-switching handling.** Marathi–English and Hindi–English code-mixing is ubiquitous. The current pipeline treats each language separately; mixed utterances may degrade.
- **Synthetic data gap.** The L2 side of training pairs is synthesised, not real (the L1 side is real per Cosy-Accent). CosyAccent’s reported gap to real-parallel training is small but **the specific 0.3% figure is unconfirmed in the paper** and has not been validated for Indic languages with different phonology.
- **ASR bottleneck.** Evaluation depends on IndicConformer / IndicWhisper quality. If IndicWhisper itself has high WER on dialect speech, the WER-before metric is unreliable.
- **Single-GPU constraint.** 32GB GPU limits model size and training scale. Results may improve with more compute, but the prototype should be feasible on consumer hardware.
- **Novelty window.** The ASR gap for dialects is closing. Move fast on the conference submission.

Execution Timeline (Two 4-Month Phases)

The work is sequenced so Paper 1 is finished and submitted before Paper 2 begins, with a clean handoff of code and artifacts.

Phase 1 — Paper 1 (Months 1–4): Method + Marathi Proof of Concept

1. Month 1 — Data + resolve the design fork:

- Download RESPIN-S1.0 Marathi + OpenSLR 103 + IndicVoices-R Marathi.
- Fine-tune IndicF5 (LoRA) on IndicVoices-R Marathi + RESPIN dialect audio.
- **Day-1 sanity check:** test Path 1 (Varhadi LoRA + X’s clean clip); if voice drifts, move to Path 2 (VANI-style conditioning).
- Run the cascaded ASR→TTS baseline on day one.
- Extract reference transcripts; set up experiment logging.

2. Month 2 — Synthetic pairs + MVP model:

- Task 1: synthesise 3,200+ (real L1 + cloned-voice L2) pairs, quality-filtered (Varhadi → Standard Pune).
- Task 2: train VQ tokenizer (512 codes, 256 dim) and AR converter (6-layer transformer).

3. Month 3 — Evaluation + ablations:

- Evaluate on RESPIN Marathi real audio (WER before/after, speaker similarity, dialect-classifier accuracy).
- Run Paper-1 ablations: VQ vs direct spectrogram; with/without GRPO; synthetic-only vs synthetic+real; with/without speaker conditioning.
- Begin per-phoneme error analysis.

4. Month 4 — Write and submit Paper 1:

- Write INTERSPEECH/ICASSP submission; release benchmark (scripts, splits, baselines).

Phase 2 — Paper 2 (Months 5–8): Scale-Up, Multilingual, Analysis

1. Month 5 — Second dialect + second language:

- South Konkan → Standard Pune (RESPIN D1 → D3).
- Awadhi Bundeli → Khariboli (RESPIN D3 → D1 Hindi) + IndicVoices-R Hindi.
- Cross-dialect transfer test: does the Varhadi-trained model partially generalise to South Konkan / Awadhi?

2. Month 6 — Multilingual model:

- Train a shared-codebook model with language/dialect-ID conditioning across Marathi + Hindi.
- If Path 1 failed in Phase 1, adopt Path 2 and compare identity preservation.

3. Month 7 — Deep ablations + analysis:

- Codebook size (256/512/1024), AR depth (3/6/12), dialect-strength interpolation.
- Cross-lingual transfer analysis; controlled disentanglement study.

4. Month 8 — Write and submit Paper 2:

- Write TASLP/CSL (or second conference) submission; extend the benchmark to Marathi + Hindi.

Expected timeline: 4 months per paper (8 months total). First WER results in Month 2; Paper 1 submitted end of Month 4; Paper 2 submitted end of Month 8.

References

- [1] G. A. Grierson. *Linguistic Survey of India*. 1907.
- [2] N. Ding et al., “TokAN: Accent Normalization Using Self-Supervised Speech Tokens,” arXiv:2607.03928, 2026. (*NOTE: verify WER figures 12.40% $\rightarrow$ 9.89% $\rightarrow$ 9.23% against the actual paper — cited draft produced a placeholder arXiv ID and ICASSP 2025 venue that do not match the real paper.*)
- [3] M. Li et al., “CosyAccent: Source-Synthesis for Accent Normalisation,” Proc. ICASSP, 2026. arXiv:2602.19166.
- [4] V. Pratap et al., “Scaling Speech Technology to 1,406 Languages,” arXiv:2305.13516, 2023.
- [5] T. Javed et al., “IndicVoices: Towards Building an Inclusive Multilingual Speech Dataset,” arXiv:2403.01926, 2024.
- [6] AI4Bharat, “IndicVoices-R: A Massive Multilingual Multi-speaker Speech Corpus for Scaling Indian TTS,” NeurIPS 2024.
- [7] M. Choudhury et al., “RASMALAI: Large-Scale Speech Dataset with Rich Text Descriptions for 23 Indian Languages,” arXiv:2505.18609, 2025.
- [8] AI4Bharat, “IndicParlerTTS: Text-Description-Guided TTS for Indian Languages,” arXiv:2505.18610, 2025.
- [9] OpenSLR, “SLR103: Marathi Read Speech Corpus,” <https://openslr.org/103/>.
- [10] OpenSLR, “SLR64: Google Marathi Speech Corpus,” <https://openslr.org/64/>.
- [11] F. He et al., “Open-source Multi-speaker Speech Corpora for Building Gujarati, Kannada, Malayalam, Marathi, Tamil and Telugu Speech Synthesis Systems,” Proc. LREC, 2020.
- [12] S. Kumar et al., “RESPIN-S1.0: A Read Speech Corpus of 10,000+ Hours in Dialects of Nine Indian Languages,” NeurIPS 2025.
- [13] R. Kumar et al., “Annotated Speech Corpus for Low Resource Indian Languages: Awadhi, Bhojpuri, Braj and Magahi,” Proc. S4SG, 2022.
- [14] S. K. Chatterji, *The Origin and Development of the Bengali Language*. Calcutta University Press, 1926.
- [15] N. Sultana et al., “ONUBAD: A Comprehensive Dataset for Automated Conversion of Bangla Regional Dialects into Standard Bengali,” Data in Brief, vol. 58, 2025.
- [16] R. Annamdevula et al., “CrossAccent-TTS: Cross-Lingual Accent-Intensity Controllable Text-to-Speech via Disentangled Speaker and Accent Representations,” Proc. INTERSPEECH, 2026.
- [17] Y. Guo et al., “MulliVC: Multi-lingual Voice Conversion With Cycle Consistency,” arXiv:2408.04708, 2024.
- [18] Q. Bai et al., “Accent Normalisation Using Self-Supervised Discrete Tokens with Non-Parallel Data,” Proc. INTERSPEECH, 2025.
- [19] T. Javed et al., “LAHAJA: A Robust Multi-accent Benchmark for Evaluating Hindi ASR Systems,” arXiv:2408.11440, 2024.

- [20] S. Pulikodan et al., “VAANI: Capturing the Language Landscape for an Inclusive Digital India,” arXiv:2603.28714, 2026.
- [21] “Optimized Multilingual TTS: Accents & Emotions for Indic Languages,” 2025.
- [22] NVIDIA, “VANI: Accent-, Language-, and Speaker-Controllable TTS with Disentangled Embeddings,” 2024. (*Reference architecture for Path 2: independent accent/language/speaker conditioning.*)
- [23] AI4Bharat, “IndicF5: Reference-Audio-Cloning TTS for Indian Languages,” Hugging Face model tree (0.4B, flow-matching; 10+ community fine-tunes).
- [24] A. Mishra et al., “Language Translation and Change of Accent for Speech-to-Speech Task Using Diffusion Model,” arXiv:2505.04639, 2025.